

System and Devices (SYS3)

Lecture Review – Week 10

Dr Pengcheng Liu

Welcome



Check-In Code:

878321

Log into checkin.york.ac.uk, find the session and enter the code.

Valid until Thursday, 30 April 2026 11:00



UNIVERSITY
of York

Week 10 Module Overview and Updates



Teaching
Excellence
Framework

Module Overview and Updates



- **Week 10:**

Lecture: In Week 10, our discussion is focused on some cache memory optimization techniques in Lecture 17 and the 4th Tutorial Lecture on Caches.

Lab: In Week 10 there will be no algorithm coding. However we will use the pipeline toolset to explore some tradeoffs relating to power consumption of the CPU and the effect of optimization.

Exercises/Quizzes:

Exercise 3 will be discussed in the Lecture Review session.

Module Overview and Updates



- **Next Week (Week 11):**

Revision week

The lab sessions and the lecture review session are optional Q&As.
Check-in will not be observed.



UNIVERSITY
of York

Questions on [Padlet](#)

Padlet Questions

S2 Week 7 L12 VLIW Processor

how does a VLIW processor handle when there is a jump instruction?

A VLIW (Very Long Instruction Word) processor relies on static scheduling by the compiler, meaning:

- The compiler decides instruction order and parallelism ahead of time
- The hardware does NOT perform hazard detection or dynamic scheduling

Handling a jump (branch) instruction:

- A branch is treated as just another operation in the VLIW bundle
- It typically occupies the integer/branch functional unit slot
- Since there is no dynamic control hazard handling, the compiler must manage control flow

Bundle

120



40 40 40



VLIW



Load

ALU

FP

Padlet Questions

S2 Week 7 L12 VLIW Processor

how does a VLIW processor handle when there is a jump instruction?

Key implications:

- The compiler must: Predict or schedule around the branch; Insert NOPs (stalls) if necessary; Use techniques like loop unrolling and software pipelining
- If the branch outcome is uncertain: Instruction slots after the branch may be wasted; This leads to poor slot utilization
- Unlike superscalar processors: No branch prediction hardware; No speculative execution

In a VLIW processor, a jump/branch is statically scheduled by the compiler and occupies a functional unit slot. The compiler must handle control hazards explicitly (e.g., inserting stalls or reordering instructions), as there is no hardware-based branch prediction or dynamic hazard detection.

Padlet Questions

S2 Week 7 L12 Multithreading vs Hyperthreading

can you go into details between multithreading and hyperthreading. I can't really differentiate the processes

Multithreading allows multiple threads to share the functional units of a single processor in an overlapping fashion.

- A single CPU runs multiple threads
- Threads take turns using the CPU resources

Types of multithreading:

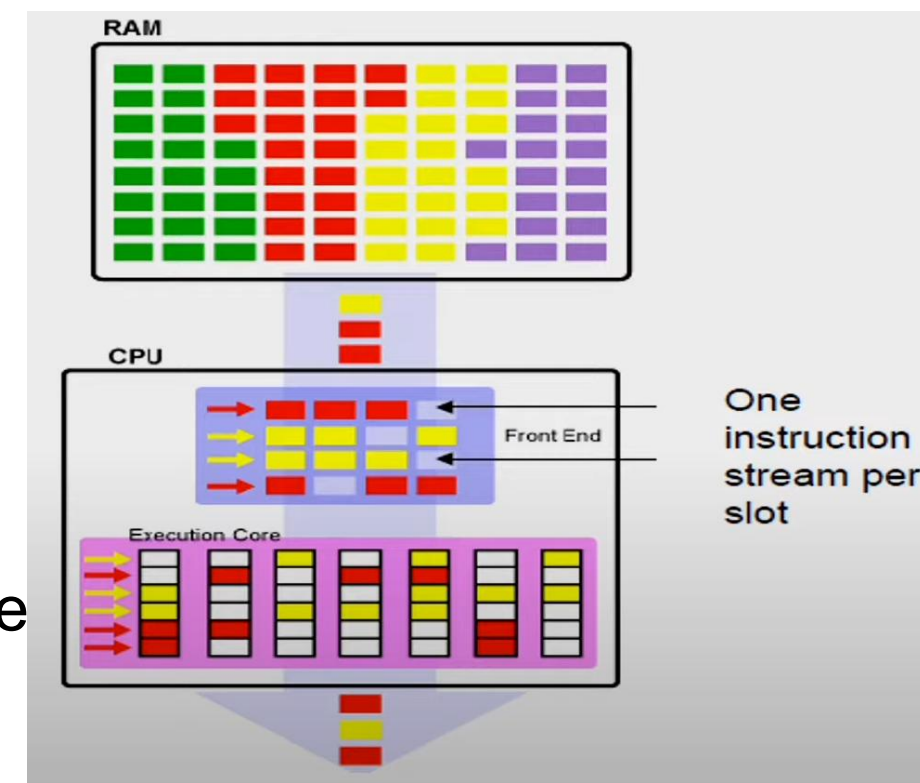
Coarse-grained multithreading

- Switch threads only on long stalls (e.g., cache miss)
- Simple but may leave CPU idle

Fine-grained multithreading

- Switch threads every cycle
- Keeps pipeline busy but reduces single-thread performance

Executes multiple threads by switching between them to hide latency.



Padlet Questions

S2 Week 7 L12 Multithreading vs Hyperthreading

can you go into details between multithreading and hyperthreading. I can't really differentiate the processes

Hyperthreading (SMT – Simultaneous Multithreading)

- A single core executes instructions from multiple threads simultaneously
- Multiple threads share execution units at the same time

Key idea:

Instead of switching threads → run them together in the same cycle

Executes multiple threads simultaneously on a single core by sharing execution units.



Padlet Questions

S2 Week 7 L12 Multithreading vs Hyperthreading

can you go into details between multithreading and hyperthreading. I can't really differentiate the processes

Key Differences

Feature	Multithreading	Hyperthreading (SMT)
Execution style	One thread at a time (interleaved)	Multiple threads at the same time
Resource usage	Shared over time	Shared simultaneously
Goal	Hide latency (e.g., memory stalls)	Maximise utilisation of idle units
Complexity	Lower	Higher (needs complex scheduling)
Pipeline usage	One thread per cycle	Multiple threads per cycle

Padlet Questions

S2 Week 8 L14 Page Replacement

How does the operating system determine which pages to swap out to disk when physical memory is full?

When physical memory becomes full and there is a need to swap out pages to disk in order to make room for new pages, the operating system employs a **page replacement algorithm** to determine which pages should be evicted from memory. The goal of the page replacement algorithm is to select the page that will minimize the impact on system performance and maximize overall efficiency.

Page Replacement Algorithms:

Least Recently Used (LRU): This algorithm selects the page that has not been accessed for the longest period of time. It assumes that pages that have not been accessed recently are less likely to be accessed in the future.

First-In-First-Out (FIFO): This algorithm selects the page that was brought into memory earliest and has been in memory the longest. It follows a simple queue-based approach, where the oldest page is evicted first.

Padlet Questions

S2 Week 8 L14 Page Replacement

How does the operating system determine which pages to swap out to disk when physical memory is full?

Clock (Second Chance): This algorithm is an enhancement of FIFO and uses a circular queue (the "clock") to keep track of page accesses. It gives each page a "second chance" by checking a reference bit associated with each page. If the reference bit is set, indicating that the page has been accessed recently, the page is given another chance and the reference bit is cleared. Otherwise, the page is evicted.

Least Frequently Used (LFU): This algorithm selects the page with the fewest number of accesses. It keeps track of the access frequency of each page and evicts the page with the lowest access count.

Most Recently Used (MRU): This algorithm selects the page that has been accessed most recently. It assumes that pages accessed most recently are more likely to be accessed again in the near future.

Padlet Questions

S2 Week 8 L14 Page Replacement

How does the operating system determine which pages to swap out to disk when physical memory is full?

Page Access Tracking:

The operating system **keeps track of page accesses** using various techniques, such as maintaining a timestamp for each page access or setting a reference bit associated with each page.

When a page is accessed, the operating system updates the access information accordingly. This information is used by the page replacement algorithm to make eviction decisions.

Padlet Questions

S2 Week 8 L14 Page Replacement

How does the operating system determine which pages to swap out to disk when physical memory is full?

Eviction Decision:

When physical memory is full and a new page needs to be brought into memory, the page replacement algorithm is invoked to select the page to be evicted.

The page replacement algorithm analyzes the access information for each page and selects the page that best fits its eviction criteria (e.g., least recently used, least frequently used).

Padlet Questions

S2 Week 8 L14 Page Replacement

How does the operating system determine which pages to swap out to disk when physical memory is full?

Page Eviction:

Once the page to be evicted is selected, the operating system writes the contents of the page to disk, freeing up the corresponding page frame in physical memory.

The page table is updated to reflect that the page is no longer in physical memory and is now stored on disk.

Padlet Questions

S2 Week 8 L13 Virtual Memory

What is thrashing in virtual memory systems, and how can it be mitigated?

Thrashing is a situation in virtual memory systems where **the system spends more time swapping pages between physical memory and disk** than executing useful work, resulting in severely degraded system performance. Thrashing occurs when the system is **overcommitted**, meaning that the demand for memory exceeds the available physical memory and the system spends a significant amount of time swapping pages in and out of memory to fulfill memory requests.

How thrashing can be mitigated?

One of the most effective ways to mitigate thrashing is to **increase the amount of physical memory** in the system. By providing more physical memory, the system has more space to hold active processes and their associated pages, reducing the need for frequent swapping.

Padlet Questions

S2 Week 8 L13 Virtual Memory

What is thrashing in virtual memory systems, and how can it be mitigated?

The choice of page replacement algorithm can significantly impact thrashing. Algorithms such as Least Recently Used (LRU) or Clock (Second Chance) are better suited for mitigating thrashing as they prioritize evicting pages that are less likely to be accessed in the near future. Optimising the page replacement algorithm can help reduce unnecessary page swaps and alleviate thrashing.

Operating systems typically provide parameters that control the behaviour of virtual memory management, such as the size of the page file or swap space and the threshold for triggering page swaps. Adjusting these parameters based on system workload and resource availability can help mitigate thrashing. For example, increasing the size of the page file or swap space can provide more room for swapping pages, reducing the likelihood of thrashing.

Padlet Questions

S2 Week 8 L13 Virtual Memory

What is thrashing in virtual memory systems, and how can it be mitigated?

Thrashing often occurs when the system attempts to run more processes than the available physical memory can accommodate. **Optimising process scheduling algorithms** to prioritise CPU time for processes with high memory demands or reducing the number of concurrent processes running can help prevent thrashing.

Memory leaks, where a program continuously allocates memory without releasing it, can exacerbate thrashing by consuming excessive memory resources. Identifying and fixing memory leaks in applications can help prevent unnecessary memory consumption and reduce the likelihood of thrashing.

Monitoring system performance metrics such as page fault rate, disk I/O activity, and CPU utilization can help identify signs of thrashing early on. By proactively monitoring system performance, administrators can take corrective actions to mitigate thrashing before it severely impacts system performance.

Padlet Questions

S2 Week 10 TL4 - Index and Offset Calculation:

The first address:

Index bits = 1001100110 (binary). Convert to decimal: $1 \cdot 2^9 + 0 \cdot 2^8 + 0 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 614$, Thus, Set = 614

Block offset bits = 000100. Convert to decimal: $0 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 = 4$

Since each word is 4 bytes, the word offset inside the block is:

Word offset = Block offset / 4 = $4 / 4 = 1$

Thus: Word inside block = Word 1

Index and Offset Calculations

A cache has 512 KB capacity, 4B word, 64B block size and 8-way set associative. The system is using 32 bit address.

(a) 0X ABC89984 (b) 0X 485669AC

sets = $CS / (BSXA) = 2^{19} / (2^6 \times 2^3) = 2^{10} = 1024$ sets

1 word = 4B, Hence 64 byte block has 16 words.

Tag = 16	Index = 10	Offset = 6 (4+2)
----------	------------	------------------

0X ABC89984 = 1001 1001 1000 0100 → Set 614, word 1

0X 485669AC = 0110 1001 1010 1100 → Set 422, word 11

Padlet Questions

S2 Week 10 TL4 - Index and Offset Calculation:

The second address:

Index bits = 0110100110 (binary). Convert to decimal:

$$0 \cdot 2^9 + 1 \cdot 2^8 + 1 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 422, \text{ Thus, Set} = 422$$

Block offset bits = 101100. Convert to decimal: $1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 = 44$

Since each word is 4 bytes, the word offset inside the block is:

$$\text{Word offset} = \text{Block offset} / 4 = 44 / 4 = 11$$

Thus: Word inside block = Word 11

Index and Offset Calculations

A cache has 512 KB capacity, 4B word, 64B block size and 8-way set associative. The system is using 32 bit address.

(a) 0X ABC89984 (b) 0X 485669AC

$$\# \text{ sets} = \text{CS} / (\text{BSXA}) = 2^{19} / (2^6 \times 2^3) = 2^{10} = 1024 \text{ sets}$$

1 word = 4B, Hence 64 byte block has 16 words.

Tag = 16	Index = 10	Offset = 6 (4+2)
----------	------------	------------------

0X ABC89984 = 1001 1001 1000 0100 → Set 614, word 1

0X 485669AC = 0110 1001 1010 1100 → Set 422, word 11

Padlet Questions

S2 Week 9 L15 - Cache intro 2 Indexes in Fully Associative Caches

Am I correct in understanding that in a fully associative cache, there is no index, since every block can reside anywhere in the cache?

In a fully associative cache, there is no index used in the traditional sense because **any memory block can be stored in any cache line.**

- In direct-mapped and set-associative caches, the memory address is split into tag, index, and block offset.
- The index is used to quickly locate the specific set or line in the cache where a block might reside.
- But in a fully associative cache, there is **only one set** (effectively), and the block can be placed anywhere.

Since there's no index to narrow down the search, the system performs a parallel comparison of the tag against all cache lines.

This allows any block to be placed in any location, but requires more complex and expensive hardware to do all those comparisons at once.

Padlet Questions

S2 Week 9 L16 - Block replacement 2 Types of Misses

Please can you clarify the situations that cause Capacity and Conflict Misses? The points given on slide 20 seem a little loose, for example, Capacity Misses are bullet pointed as "If too many blocks map to the same set", but presumably this also applies in a direct mapped cache, where sets do not apply?

Capacity misses

- A capacity miss happens when the **cache does not have enough total space** to hold all the blocks needed for a program, even if associativity were ideal (e.g., even in a fully associative cache).
- These misses occur when the working set (the actively used data) is larger than the cache itself.

Capacity misses are **about cache size limitations**, not how blocks are placed.

Padlet Questions

S2 Week 9 L16 - Block replacement 2 Types of Misses

Please can you clarify the situations that cause Capacity and Conflict Misses? The points given on slide 20 seem a little loose, for example, Capacity Misses are bullet pointed as "If too many blocks map to the same set", but presumably this also applies in a direct mapped cache, where sets do not apply?

Conflict Misses

- A conflict miss occurs in **set-associative** or **direct-mapped caches** when **multiple blocks compete for the same set or line**, even though the cache has enough total space.
- Applies to direct direct-mapped and set-associative caches.

Padlet Questions

S2 Week 10 L17 - Inclusive Caches: I am not 100% sure what it means when you talk about a core request that comes in an inclusive hierarchy. You explained this in the lecture but I did not really understand what you meant was happening. Would you be able to describe what happens in this case again to help me understand? Thanks

When a cache miss occurs, the cache controller first checks if the requested data is present in any of the lower-level caches. If the data is not found in the lower-level caches, the cache controller fetches the data from the next level of the cache hierarchy or from the main memory. If the requested data is found in a lower-level cache, the cache controller retrieves the data from that cache and also copies it to the higher-level cache. **In an inclusive cache, all the data in the lower-level caches is also present in the higher-level caches**, so the data can be copied to the higher-level cache without any additional coherence operations. If the cache line containing the requested data is already present in the higher-level cache, the copy operation is not necessary. The cache controller can simply retrieve the data from the higher-level cache and return it to the requesting core.

Padlet Questions

S2 Week 10 L17 - Higher Associativity Hit Time Impact: *Can't remember exactly where this was stated but we're told that increasing the associativity of a cache would increase the hit time because more tags would have to be examined. At a later point it talks about being able to examine tags in parallel so how does an increasing amount of tags to examine impact the time to check them?*

Tags can be compared in parallel across all ways of a set-associative cache. However, parallel tag comparison requires multiple comparators, one per way. More associativity means more comparators needed and thus more circuit complexity, wiring, power usage and path length. Even though comparisons occur in parallel, the added logic and complexity increase the overall delay, thus increasing the effective hit time.

Padlet Questions

S2 Week 9 L15 - Cache Memory:

Explain the difference between a compulsory, capacity, and conflict cache miss.

The terms compulsory, capacity, and conflict cache miss refer to different scenarios in which cache misses occur in computer systems. These types of cache misses help to categorize and analyze the reasons behind cache inefficiencies.

Compulsory Cache Miss: Also known as a cold miss, occurs when an item is **accessed for the first time** and **is not present in the cache**.

Compulsory misses are unavoidable and occur regardless of the cache size because there is no previous opportunity to load the data into the cache.

These misses typically happen when a program starts execution or when data is accessed for the first time after a cache has been cleared or initialized.

Increasing the cache size will not eliminate compulsory misses, but it can reduce the impact of subsequent misses by storing more data in the cache.

Padlet Questions

S2 Week 9 L15 - Cache Memory:

Explain the difference between a compulsory, capacity, and conflict cache miss.

Capacity Cache Miss: it occurs when the **cache is too small** to hold all the required data, and **some items must be evicted** to make space for new ones. Capacity misses are primarily caused by **limitations in cache size** relative to the working set size of the program.

These misses often occur when there is a large amount of data or code that needs to be stored in the cache, but the cache size is insufficient to accommodate all of it.

Increasing the cache size can reduce the occurrence of capacity misses by providing more storage space for frequently accessed data and instructions.

Padlet Questions

S2 Week 9 L15 - Cache Memory:

Explain the difference between a compulsory, capacity, and conflict cache miss.

Conflict Cache Miss: Also known as a collision miss, occurs in set-associative or direct-mapped caches when **multiple memory blocks map to the same cache set**.

Conflict misses are caused by cache organization and occur **when different memory blocks contend for the same cache set**, leading to eviction of existing cache lines.

These misses are more likely to occur in caches with limited associativity (such as direct-mapped caches) or when the number of cache sets is small relative to the number of memory blocks.

Increasing cache associativity or reducing the cache set size can reduce the occurrence of conflict misses by providing more flexibility in cache placement decisions.



UNIVERSITY
of York

SYS3 Exercises/Quizzes Discussions

Exercise 3

Exercise 3

Q1. Why is virtual memory a concern for L1 cache performance? How can the performance penalty be reduced for cache accesses?

Virtual memory can impact L1 cache performance due to its interaction with the memory hierarchy.

When virtual memory is **utilised heavily**, the system may need to **swap data frequently** between RAM and disk. This can lead to **cache thrashing**, where cache lines are constantly being replaced with new data from memory or disk, reducing the effectiveness of the cache. As a result, the CPU may waste cycles fetching data from slower memory levels or even from disk, leading to degraded performance.

Besides, **accessing data from disk is significantly slower** than accessing data from RAM or cache. When data needs to be fetched from disk due to virtual memory swapping, it introduces increased latency for cache accesses. This latency can degrade L1 cache performance, especially for applications with high memory demands.

To reduce the performance penalty for cache accesses in the presence of virtual memory, **several strategies** can be employed:

Exercise 3

Q1. Why is virtual memory a concern for L1 cache performance? How can the performance penalty be reduced for cache accesses?

Efficient memory management practices can help minimise the need for virtual memory swapping. This includes **optimising data structures and algorithms** to reduce memory usage, minimizing memory fragmentation, and prioritizing data locality to improve cache utilization.

Larger cache sizes can help mitigate the impact of virtual memory by storing more data closer to the CPU. This reduces the frequency of cache misses and the need to fetch data from slower memory levels or disk.

Prefetching data into the cache based on predictable access patterns can help reduce the latency introduced by virtual memory accesses. By fetching data proactively before it is actually needed by the CPU, prefetching can help hide the latency of accessing data from slower memory levels.

Using cache replacement policies that prioritise keeping frequently accessed data in the cache can help reduce the impact of cache thrashing caused by virtual memory.

Exercise 3

Q2. Consider the following page reference string:

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6.

How many page faults occur for the following page replacement algorithms assuming one, two, three, four, five, six, or seven frames? Assume that all pages are initially used by pages left over from another process, so your first unique pages will all cost one fault each.

- a) LRU replacement
- b) FIFO replacement
- c) Optimal replacement

Exercise 3

FIFO:

3 frames:

	1,	2,	3,	4,	2,	1,	5,	6,	2,	1,	2,	3,	7,	6,	3,	2,	1,	2,	3,	6
Frame 1:	1	1	1	4		4	4	6	6	6		3	3	3	–	2	2	–	2	6
Frame 2:		2	2	2	–	1	1	1	2	2	–	2	7	7		7	1		1	1
Frame 3:			3	3		3	5	5	5	1		1	1	6		6	6		3	3

16 page faults

4 frames:

	1,	2,	3,	4,	2,	1,	5,	6,	2,	1,	2,	3,	7,	6,	3,	2,	1,	2,	3,	6
Frame 1:	1	1	1	1		–	5	5	5	5		3	3	3	–	3	1		1	
Frame 2:		2	2	2	–		2	6	6	6		6	7	7		7	7		3	
Frame 3:			3	3			3	3	2	2	–	2	2	6		6	6		6	–
Frame 4:				4			4	4	4	1		1	1	1		2	2	–	2	

14 page faults

First come, first serve. Whatever request first will have some sort of preference or whichever page has been given a frame first will be the first one to leave the memory. We check what is the first page that has been allocated and that is the one that will be ejected whenever we have a page fault.

Exercise 3



UNIVERSITY
of York

5 frames:

		1,	2,	3,	4,	2,	1,	5,	6,	2,	1,	2,	3,	7,	6,	3,	2,	1,	2,	3,	6
Frame 1:	1	1	1	1		-	1	6		6	6	6	6	-							-
Frame 2:		2	2	2	-		2	2	-	1	1	1	1					-			
Frame 3:			3	3			3	3		3	2	2	2			-			-		
Frame 4:				4			4	4		4	4	3	3		-					-	
Frame 5:							5	5		5	5	5	7								

10 page faults

6 frames:

		1,	2,	3,	4,	2,	1,	5,	6,	2,	1,	2,	3,	7,	6,	3,	2,	1,	2,	3,	6
Frame 1:	1	1	1	1		-	1	1		-				7				7	7	7	
Frame 2:		2	2	2	-		2	2	-		-			2		-		1	1	1	
Frame 3:			3	3			3	3				-		3		-		3	2	2	
Frame 4:				4			4	4						4				4	4	3	
Frame 5:							5	5						5				5	5	5	
Frame 6:								6						6	-			6	6	6	-

10 page faults

Exercise 3

Optimal:

3 frames:

	1,	2,	3,	4,	2,	1,	5,	6,	2,	1,	2,	3,	7,	6,	3,	2,	1,	2,	3,	6
Frame 1:	1	1	1	1	-	-	1	1	-	-	-	3	3	-	3	3	-	-	3	
Frame 2:	-	2	2	2	-	-	2	2	-	-	-	2	7	-	-	2	2	-	-	2
Frame 3:	-	-	3	4	-	-	5	6	-	-	-	6	6	-	-	6	1	-	-	6

11 page faults

4 frames:

	1,	2,	3,	4,	2,	1,	5,	6,	2,	1,	2,	3,	7,	6,	3,	2,	1,	2,	3,	6
Frame 1:	1	1	1	1	-	-	1	1	-	-	-	-	7	-	-	-	1	-	-	-
Frame 2:	-	2	2	2	-	-	2	2	-	-	-	-	2	-	-	-	2	-	-	-
Frame 3:	-	-	3	3	-	-	3	3	-	-	-	-	3	-	-	-	3	-	-	-
Frame 4:	-	-	-	4	-	-	5	6	-	-	-	-	6	-	-	-	6	-	-	-

8 page faults

Best possible replacement policy: replace page that will not be used for longest period of time. It will know what will happen in the future, how the process will behave and what is the order that the pages will be requested. We will replace the page that is going to be used the furthest in the future or not at all, maybe it will never be used again.

Exercise 3

5 frames:

	1,	2,	3,	4,	2,	1,	5,	6,	2,	1,	2,	3,	7,	6,	3,	2,	1,	2,	3,	6
Frame 1:	1	1	1	1		-	1	1		-			1	-			-			
Frame 2:		2	2	2	-		2	2	-		-		2			-		-		
Frame 3:			3	3			3	3				-	3		-				-	
Frame 4:				4			4	4					7							
Frame 5:							5	6					6	-						-

7 page faults, which is the best we can do

Exercise 3

LRU:

3 frames:

	1,	2,	3,	4,	2,	1,	5,	6,	2,	1,	2,	3,	7,	6,	3,	2,	1,	2,	3,	6
Frame 1:	1	1	1	4		4	5	5	5	1		1	7	7		2	2	-		2
Frame 2:		2	2	2	-	2	2	6	6	6		3	3	3	-	3	3		-	3
Frame 3:			3	3		1	1	1	2	2	-	2	2	6		6	1			6

15 page faults

4 frames:

	1,	2,	3,	4,	2,	1,	5,	6,	2,	1,	2,	3,	7,	6,	3,	2,	1,	2,	3,	6
Frame 1:	1	1	1	1		-	1	1		-		1	1	6			6			-
Frame 2:		2	2	2	-		2	2	-		-	2	2	2		-	2	-		
Frame 3:			3	3			5	5				3	3	3	-		3		-	
Frame 4:				4			4	6				6	7	7			1			

10 page faults

Least recently used: used past knowledge, rather than future, replace page that has not been used for the longest period of time; must keep track of time of last use for each page.

We will check the one that hasn't been accessed for the longest period of time.

In this case, when 2 is accessed, there is no page fault, but we reset the timer for 2 as two has been accessed recently, and 2 was the most recent accessed page.

Exercise 3



5 frames:

	1,	2,	3,	4,	2,	1,	5,	6,	2,	1,	2,	3,	7,	6,	3,	2,	1,	2,	3,	6
Frame 1:	1	1	1	1	-	-	1	1	-	-	-	1	1	-	-	-	-	-	-	-
Frame 2:	-	2	2	2	-	-	2	2	-	-	-	2	2	-	-	-	-	-	-	-
Frame 3:	-	-	3	3	-	-	3	6	-	-	-	6	6	-	-	-	-	-	-	-
Frame 4:	-	-	-	4	-	-	4	4	-	-	-	3	3	-	-	-	-	-	-	-
Frame 5:	-	-	-	-	-	-	5	5	-	-	-	5	7	-	-	-	-	-	-	-

8 page faults

6 frames:

	1,	2,	3,	4,	2,	1,	5,	6,	2,	1,	2,	3,	7,	6,	3,	2,	1,	2,	3,	6
Frame 1:	1	1	1	1	-	-	1	1	-	-	-	-	1	-	-	-	-	-	-	-
Frame 2:	-	2	2	2	-	-	2	2	-	-	-	-	2	-	-	-	-	-	-	-
Frame 3:	-	-	3	3	-	-	3	3	-	-	-	-	3	-	-	-	-	-	-	-
Frame 4:	-	-	-	4	-	-	4	4	-	-	-	-	4	-	-	-	-	-	-	-
Frame 5:	-	-	-	-	-	-	5	5	-	-	-	-	7	-	-	-	-	-	-	-
Frame 6:	-	-	-	-	-	-	-	6	-	-	-	-	6	-	-	-	-	-	-	-

7 page faults, which is the fewest we can have with 7 pages

Exercise 3

Q3. Consider a demand-paging system with the following time-measured utilizations:

CPU utilization 20%

Paging disk 97.7%

Other I/O devices 5%

For each of the following, say whether it will (or is likely to) improve CPU utilization. Explain your answers.

- (a) Install a faster CPU.
- (b) Install a bigger paging disk.
- (c) Increase the degree of multiprogramming.
- (d) Decrease the degree of multiprogramming.
- (e) Install more main memory.
- (f) Install a faster hard disk or multiple controllers with multiple hard disks.
- (g) Add pre-paging to the page fetch algorithms.
- (h) Increase the page size.

Exercise 3

(a) Install a faster CPU.

Installing a faster CPU will **decrease** CPU utilisation because a faster CPU will process its instructions more quickly than the old CPU, leaving more idle time.

(b) Install a bigger paging disk.

Installing a bigger paging disk will have no particular effect on the system. It's being used to 97.7% of time capacity, not space capacity.

Exercise 3

(c) Increase the degree of multiprogramming

Increasing the degree of multiprogramming will **decrease locality** and therefore **increase page faults**, causing the paging disk to max out and CPU utilisation to go down. On the other hand, if the newly introduced programs have small memory footprints and do little I/O, they will help increase CPU utilisation. In other words, the effect of increasing multiprogramming depends entirely on the character of the new programs.

(d) Decrease the degree of multiprogramming

Exactly the opposite as part c.

Exercise 3

(e) Install more main memory

Install more main memory—Likely to improve CPU utilisation as more pages can remain resident and not require paging to or from the disks.

(f) Install a faster hard disk or multiple controllers with multiple hard disks

Install a faster hard disk or multiple controllers with multiple hard disks—Also an improvement, for as the disk bottleneck is removed by faster response and more throughput to the disks, the CPU will get more data more quickly.

Exercise 3

(g) Add pre-paging to the page fetch algorithms

Add prepaging to the page fetch algorithms—Again, the CPU will get more data faster, so it will be more in use. This is only the case if the paging action is amenable to prefetching (i.e., some of the access is sequential)

(h) Increase the page size

Increase the page size—Increasing the page size will result in fewer page faults if data is being accessed sequentially. If data access is more or less random, more paging action could ensue because fewer pages can be kept in memory and more data is transferred per page fault. So this change is as likely to decrease utilisation as it is to increase it.

Exercise 3

Q4. The average memory-access time is determined by three factors. List these factors, and for each, name a technique that reduces the cache-hit time by affecting that factor.

The factors are:

- (1) **cache miss-rate**, affected by using a victim cache,
- (2) **cache hit-time**, affected by using a virtually-indexed and physically-tagged cache, and
- (3) **cache miss-penalty**, affected by giving priorities to reads over writes.

Exercise 3

Q5. Which cache mapping function does not require a replacement algorithm?

- (A) Direct mapping
- (B) Set associative mapping
- (C) Fully associative mapping
- (D) All above

(A) Direct mapping

Direct mapping is the cache mapping function that **does not require a replacement** algorithm.

In direct mapping, each block of main memory can be mapped to only one specific cache line. Therefore, when a new block needs to be brought into the cache and the cache line it maps to is already occupied, the existing block in that cache line is replaced without the need for a replacement algorithm.

In set associative mapping and fully associative mapping, multiple blocks of main memory can be mapped to the same cache set or the entire cache, respectively. In these cases, a replacement algorithm is necessary to determine which block to evict from the cache when a new block needs to be brought in and the set or cache is already full.

Exercise 3



Q6. Which cache write mechanism creates more bus traffic?

- (A) Write through
- (B) Write back
- (C) Both
- (D) Neither

(A) Write through

Write through cache write mechanism creates more bus traffic compared to write back.

In the write-through cache policy, **every write operation to the cache is immediately propagated to both the cache and main memory**. This means that for every write operation, there are **two bus transactions**: one to update the cache and another to update main memory. Therefore, write-through cache policies result in more bus traffic because each write operation requires communication with both the cache and main memory.

In contrast, in the write-back cache policy, writes are initially only made to the cache. The corresponding block in main memory is only updated when the block is evicted from the cache. This means that write-back cache policies can potentially reduce bus traffic because not every write operation requires communication with main memory.

Exercise 3

Q7. Which cache write mechanism allows an updated memory location in the cache to remain out of date in memory until the block containing the updated memory location is replaced in the cache?

- (A) Write through
- (B) Write back
- (C) Both
- (D) Neither

(B) Write back

In the write-back cache write mechanism, an updated memory location in the cache can remain out of date in memory until the block containing the updated memory location is replaced in the cache.

In the write-back policy, when a location in the cache is modified, the corresponding location in main memory is not immediately updated. Instead, the modification is kept only in the cache. The updated data is written back to main memory only when the cache block is replaced, ensuring consistency between the cache and main memory.

In contrast, in the write-through cache policy (option A), every write operation to the cache is immediately propagated to both the cache and main memory, ensuring that main memory is always up-to-date with the data in the cache.

Exercise 3

Q8. Which cache write mechanism best supports bus watching for use with multi-processor systems?

- (A) Write through
- (B) Write back
- (C) Both
- (D) Neither

(A) Write through

Write-through cache write mechanism best supports bus watching for use with multi-processor systems.

In a write-through cache policy, every write operation to the cache is immediately propagated to both the cache and main memory. This ensures that the data in main memory is always up-to-date with the data in the cache. As a result, other processors in a multi-processor system can monitor the system bus to observe memory updates initiated by write operations from other processors.

In contrast, in a write-back cache policy, writes are initially only made to the cache, and the corresponding block in main memory is only updated when the block is evicted from the cache. This delayed updating of main memory can make it more challenging for other processors to observe memory updates using bus watching techniques.

Exercise 3

Q9. A processor has a 32-bit memory address space (i.e. 32-bit addresses). The memory is broken into blocks of 32 bytes each. The computer also has a cache capable of storing 16K bytes,

- (1) How many blocks can the cache store?
- (2) Assuming the cache uses direct-mapping, how many bits are there in each of the Tag, Index and Byte Offset fields of the address. Show your calculations.
- (3) Assuming the cache uses a 4-way set-associative mapping, how many bits are there in each of the Tag, Index and Byte Offset fields of the address. Show your calculations.

(1)

To find out how many blocks the cache can store, we need to determine the number of blocks the cache can hold based on its size and the size of each block.

Given: Cache size = 16 KB = $16 * 1024$ bytes Block size = 32 bytes

Number of blocks the cache can store = Cache size / Block size = $(16 * 1024)$ bytes / 32 bytes = 512 blocks

So, the cache can store 512 blocks.

Exercise 3

Q9. A processor has a 32-bit memory address space (i.e. 32-bit addresses). The memory is broken into blocks of 32 bytes each. The computer also has a cache capable of storing 16K bytes,

- (1) How many blocks can the cache store?
- (2) Assuming the cache uses direct-mapping, how many bits are there in each of the Tag, Index and Byte Offset fields of the address. Show your calculations.
- (3) Assuming the cache uses a 4-way set-associative mapping, how many bits are there in each of the Tag, Index and Byte Offset fields of the address. Show your calculations.

(2)

For direct-mapping, we need to determine the number of bits in each of the Tag, Index, and Byte Offset fields of the address.

Given: Memory address space = 32-bit addresses Block size = 32 bytes

Calculate the number of bits needed for each field:

- Number of bits for Byte Offset = $\log_2(32) = 5$ bits
- Number of blocks in the cache = 512
- Number of bits for Index = $\log_2(512) = 9$ bits
- Remaining bits for Tag = Total bits - (Index bits + Offset bits) = 32 bits - (9 bits + 5 bits) = 18 bits

So, for direct-mapping: Tag field: 18 bits; Index field: 9 bits; Byte Offset field: 5 bits

Exercise 3

Q9. A processor has a 32-bit memory address space (i.e. 32-bit addresses). The memory is broken into blocks of 32 bytes each. The computer also has a cache capable of storing 16K bytes,

- (1) How many blocks can the cache store?
- (2) Assuming the cache uses direct-mapping, how many bits are there in each of the Tag, Index and Byte Offset fields of the address. Show your calculations.
- (3) Assuming the cache uses a 4-way set-associative mapping, how many bits are there in each of the Tag, Index and Byte Offset fields of the address. Show your calculations.

(3)

For a 4-way set-associative mapping, we need to determine the number of bits in each of the Tag, Index, and Byte Offset fields of the address.

Given: Memory address space = 32-bit addresses Block size = 32 bytes Number of sets in the cache = $512 / 4 = 128$ sets

Calculate the number of bits needed for each field:

- Number of bits for Byte Offset = $\log_2(32) = 5$ bits
- Number of sets in the cache = 128
- Number of bits for Index = $\log_2(128) = 7$ bits
- Remaining bits for Tag = Total bits - (Index bits + Offset bits) = 32 bits - (7 bits + 5 bits) = 20 bits

So, for 4-way set-associative mapping: Tag field: 20 bits; Index field: 7 bits; Byte Offset field: 5 bits



UNIVERSITY
of York

Discussion Forum

Discussion Forum - Key Themes for Discussion



1. Hardware-Based Speculation and Reorder Buffers

Understanding hardware-based speculation, including the three key ideas that enable it. The role of the reorder buffer (ROB) in speculation, why it is necessary, and the specific fields in each ROB entry. Differences in how speculative execution handles stores compared to Tomasulo's algorithm.

2. Virtual Memory and Its Mechanisms

The benefits of virtual memory, such as memory isolation, efficient memory management, and support for large address spaces. How virtual memory works, including address translation and the role of the page table. Explanation of page faults and the steps taken by the operating system to handle them.

Discussion Forum - Key Themes for Discussion



3. Memory Management and Trade-Offs

The concepts of internal and external fragmentation in virtual memory and how they impact system performance.

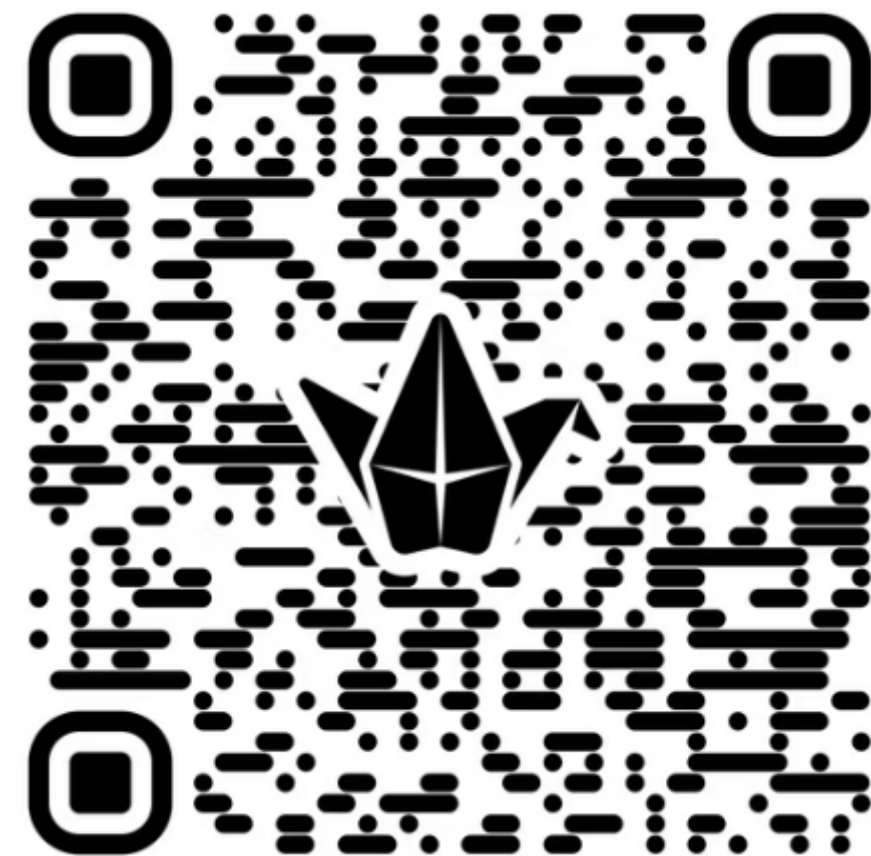
The trade-offs in choosing virtual memory page sizes, including memory overhead, fragmentation, and TLB performance considerations.

Discussion Forum - Pipeline & Compilers



Pick one or more of the following questions, discuss with your small group and add comments to the Padlet discussion board:

1. What are the three key ideas combined for hardware-based speculation?
2. What is reorder buffer and why is it needed for speculation?
3. What are the four fields of each entry in the reorder buffer?
4. What is the difference in how stores are handled in a speculative processor versus in Tomasulo's algorithm?

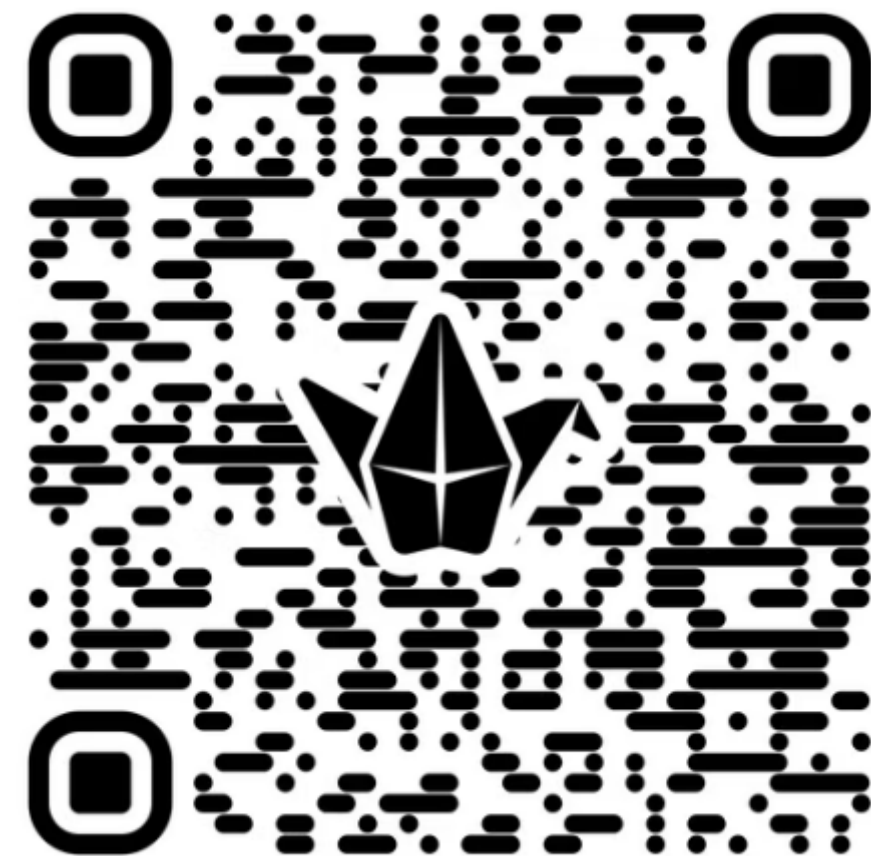


Discussion Forum - Pipeline & Compilers



Pick one or more of the following questions, discuss with your small group and add comments to the Padlet discussion board:

5. What are the benefits of using virtual memory?
6. How does virtual memory work?
7. What is the role of the page table in virtual memory?
8. What is a page fault, and how is it handled?
9. What is the difference between internal and external fragmentation in virtual memory?
10. What are the trade-offs involved in choosing the size of virtual memory pages?

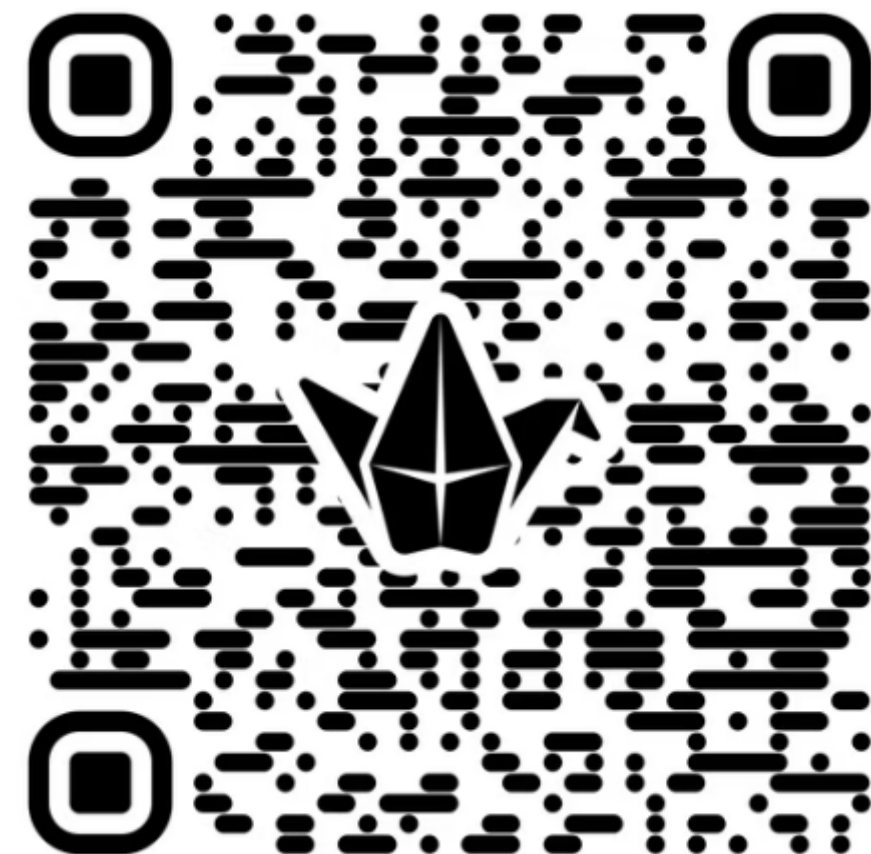


Discussion Forum - Approach



Small Group Discussions

- Form a small group with someone nearby, or feel free to move around and join your friends
- Pick one or more applications
- 15 mins group discussion
- Add comments/summary as a group to the Padlet discussion board



<https://padlet.com/pengchengliu83/week-10-u85uak92errmkvqd>

Discussion Forum

1. What are the three key ideas combined for hardware-based speculation?

- (1) **Dynamic branch prediction** to choose which instructions to execute,
- (2) **Speculation** to allow the execution of instructions before the control dependences are resolved (with the ability to undo the effects of an incorrectly speculated sequence), and
- (3) **Dynamic scheduling** to deal with the scheduling of different combinations of basic blocks.

(In comparison, dynamic scheduling without speculation only partially overlaps basic blocks because it requires that a branch be resolved before actually executing any instructions in the successor basic block.)

2. What is reorder buffer and why is it needed for speculation?

The key idea behind implementing speculation is to allow instructions to **execute out of order** but to force them to **commit in order** and to prevent any irrevocable action (such as updating state or taking an exception) until an instruction commits.

Hence, when we add speculation, we need to **separate the process of completing execution from instruction commit**, since instructions may finish execution considerably before they are ready to commit. Adding this commit phase to the instruction execution sequence requires an additional set of hardware buffers that hold the results of instructions that have finished execution but have not committed. This hardware buffer, which we call the reorder buffer, is also used to pass results among instructions that may be speculated.

2. What is reorder buffer and why is it needed for speculation?

The reorder buffer (ROB) provides additional registers in the same way as the reservation stations in Tomasulo's algorithm extend the register set.

The ROB holds the result of an instruction between the time the operation associated with the instruction completes and the time the instruction commits. Hence, the ROB is a source of operands for instructions, just as the reservation stations provide operands in Tomasulo's algorithm.

The key difference is that in Tomasulo's algorithm, once an instruction writes its result, any subsequently issued instructions will find the result in the register file. With speculation, the register file is not updated until the instruction commits (and we know definitively that the instruction should execute); thus, the ROB supplies operands in the interval between completion of instruction execution and instruction commit. The ROB is similar to the store buffer in Tomasulo's algorithm, and we integrate the function of the store buffer into the ROB for simplicity.

3. What are the four fields of each entry in the reorder buffer?

Each entry in the ROB contains four fields: **the instruction type, the destination field, the value field, and the ready field.**

The **instruction type field** indicates whether the instruction is a branch (and has no destination result), a store (which has a memory address destination), or a register operation (ALU operation or load, which has register destinations).

The destination field supplies the register number (for loads and ALU operations) or the memory address (for stores) where the instruction result should be written.

The value field is used to hold the value of the instruction result until the instruction commits.

Finally, **the ready field** indicates that the instruction has completed execution, and the value is ready.

4. What is the difference in how stores are handled in a speculative processor versus in Tomasulo's algorithm?

In Tomasulo's algorithm, a store can **update memory when it reaches write result** (which ensures that the effective address has been calculated) and the data value to store is available.

In a speculative processor, a store **updates memory only when it reaches the head of the ROB**. This difference ensures that memory is not updated until an instruction is no longer speculative.

5. What are the benefits of using virtual memory?

Virtual memory allows computers **to run programs larger than the physical memory capacity**. It provides a larger logical memory space by utilizing disk storage as an extension of RAM.

Virtual memory facilitates **running multiple programs simultaneously** by allocating a portion of virtual memory to each process. This allows for efficient multitasking without requiring each program to fit entirely in physical memory.

Virtual memory **abstracts the physical memory details** from the programmer. Programmers can write code without worrying about the physical memory limitations, making memory management easier and more flexible.

Virtual memory helps **prevent system crashes and program failures** due to out-of-memory errors. When physical memory becomes full, virtual memory systems can swap less frequently used data to disk, allowing the system to continue running without running out of memory.

5. What are the benefits of using virtual memory?

By allowing more efficient use of physical memory resources, virtual memory can **improve overall system performance**. It enables the system to keep frequently accessed data in RAM while swapping less frequently used data to disk, reducing the need for disk accesses and speeding up program execution.

Virtual memory provides **memory protection mechanisms** that prevent one process from accessing the memory of another process. Each process operates within its own virtual address space, isolated from other processes, which enhances system security and stability.

Virtual memory systems **allow multiple processes to share the same code and data pages**, reducing memory duplication and conserving memory resources. This feature is commonly used for shared libraries, which are loaded into memory once and shared among multiple processes.

6. How does virtual memory work?

Virtual memory is a memory management technique that allows a computer to use more memory than is physically available by **temporarily transferring data from random access memory (RAM) to disk storage**.

Address Translation: When a program accesses memory, it generates virtual addresses. These virtual addresses are mapped to physical addresses using a memory management unit (MMU) and a page table. The **MMU translates virtual addresses to physical addresses**, allowing the program to access the corresponding physical memory locations.

Memory Paging: Virtual memory **divides the memory space into fixed-size blocks called pages**. Similarly, physical memory is **divided into page frames of the same size**. Pages are the units of allocation in virtual memory, and page frames are the units of allocation in physical memory.

6. How does virtual memory work?

Page Fault Handling: When a program accesses a page of virtual memory that is not currently in physical memory (a page fault), the operating system interrupts the program. The operating system then retrieves the required page from disk storage and loads it into a free page frame in physical memory.

Demand Paging: Virtual memory systems typically use demand paging, a technique where **pages are loaded into physical memory only when they are accessed by a program**. This approach minimizes the amount of memory needed to run a program and reduces the initial loading time.

Page Replacement: If physical memory becomes full and there are no free page frames available, the operating system must evict a page from memory to make room for the new page. The operating system employs various page replacement algorithms, such as Least Recently Used (LRU) or First-In-First-Out (FIFO), to decide which page to evict.

Discussion Forum

6. How does virtual memory work?

Swap Space: Pages that are evicted from physical memory are temporarily stored on disk in an area called swap space. Swap space serves as an extension of physical memory and allows the system to swap pages in and out of memory as needed.

Memory Protection: Virtual memory systems provide memory protection mechanisms that prevent one process from accessing the memory of another process. Each process operates within its own virtual address space, isolated from other processes, which enhances system security and stability.

7. What is the role of the page table in virtual memory?

The page table plays a crucial role in virtual memory systems by mapping virtual addresses generated by programs to physical addresses in memory.

Address Translation: When a program accesses memory, it generates virtual addresses. These virtual addresses are not directly used to access physical memory; instead, they **must be translated to physical addresses**. The page table performs this translation.

Mapping Virtual Addresses to Physical Addresses: The page table **stores entries that map virtual addresses to corresponding physical addresses**. Each entry in the page table typically consists of a virtual page number (VPN) and a corresponding physical frame number (PFN). When the CPU generates a virtual address, the page table is consulted to determine the corresponding physical frame where the data resides.

7. What is the role of the page table in virtual memory?

Protection and Access Control: In addition to mapping addresses, the page table also stores information about access permissions and protection attributes for each page. This information includes whether the page is readable, writable, or executable, as well as any access restrictions. The page table helps enforce memory protection mechanisms to prevent unauthorized access or modification of memory.

Page Fault Handling: If a program accesses a virtual page that is not currently in physical memory, a page fault occurs. The page table is responsible for **detecting page faults and initiating the process of loading the required page** into memory from disk.

7. What is the role of the page table in virtual memory?

Page Table Management: The page table itself is typically stored in physical memory. However, because the page table can be quite large, it may not fit entirely in memory. In such cases, **the page table is divided into multiple levels** (e.g., a two-level or multi-level page table), allowing for efficient use of memory resources.

Translation Lookaside Buffer (TLB): To improve performance, modern processors often use a Translation Lookaside Buffer (TLB), a small hardware cache that **stores recently accessed page table entries**. The TLB caches translations from virtual addresses to physical addresses, reducing the need to access the full page table for every memory access.

8. What is a page fault, and how is it handled?

A page fault is an exception that occurs when a program accesses a page of virtual memory that is not currently mapped to any physical memory location. This situation typically arises when a program attempts to access a page that has been swapped out to disk or has not yet been loaded into physical memory. When a page fault occurs, the operating system must intervene to handle the exception and ensure that the required page is brought into physical memory.

Page Fault Detection: When a program attempts to access a page of virtual memory that is not currently in physical memory, the hardware detects the page fault and raises an interrupt, transferring control to the operating system.

Page Fault Handler Execution: The operating system's page fault handler is responsible for handling the page fault. This handler is part of the kernel and is executed in privileged mode.

8. What is a page fault, and how is it handled?

Determine Page Location: The page fault handler first determines the location of the required page. It checks whether the page is present in physical memory, has been swapped out to disk, or has not yet been loaded into memory.

Load Page into Memory: If the required page is not present in physical memory, the operating system initiates the process of loading the page from disk into a free page frame in physical memory. This may involve reading the page from disk into memory and updating the page table to reflect the new mapping between virtual and physical addresses.

Resume Program Execution: Once the required page is successfully loaded into memory, the page fault handler updates the page table and returns control to the program. The program's access to the virtual memory page is now satisfied, and it can proceed with its execution.

8. What is a page fault, and how is it handled?

Handling Special Cases: In some cases, the page fault may be due to an invalid memory access, such as attempting to access protected memory or accessing a page that does not exist. In such cases, the operating system may terminate the program or raise a segmentation fault error.

Optimization: To improve performance, operating systems often employ various optimizations to handle page faults efficiently. This may include techniques such as demand paging, where pages are loaded into memory only when they are accessed, or using pre-fetching strategies to anticipate future memory accesses and load pages in advance.

9. What is the difference between internal and external fragmentation in virtual memory?

Internal fragmentation and external fragmentation are two types of memory wastage that can occur in virtual memory systems.

Internal Fragmentation: Internal fragmentation occurs when **allocated memory blocks are larger than the requested size, resulting in unused memory within the allocated block.**

Cause: Internal fragmentation typically arises when memory is allocated in fixed-size units (e.g., pages or blocks), and the requested memory size does not perfectly align with these units. As a result, the allocated block may contain unused space at the end, which cannot be utilized by other processes.

Location: Internal fragmentation occurs within individual allocated memory blocks.

Effect: Internal fragmentation reduces memory utilization efficiency by wasting space within allocated blocks, leading to inefficient use of memory resources.

9. What is the difference between internal and external fragmentation in virtual memory?

External Fragmentation: External fragmentation occurs when there is enough total memory available to satisfy a request, but it is **not contiguous**, resulting in inefficient memory allocation.

Cause: External fragmentation arises when memory is allocated and deallocated over time, leading to a fragmented memory layout with free memory scattered throughout the address space. Although the total amount of free memory may be sufficient to satisfy a request, the fragmented layout makes it challenging to find a contiguous block of memory large enough to fulfill the request.

Location: External fragmentation occurs in the overall memory address space due to the fragmentation of free memory blocks.

Effect: External fragmentation reduces memory utilization efficiency by limiting the ability to allocate large contiguous blocks of memory, potentially leading to memory allocation failures even when sufficient total memory is available.

10. What are the trade-offs involved in choosing the size of virtual memory pages?

The choice of virtual memory page size involves trade-offs between page table size, memory utilization, TLB efficiency, I/O operations, and memory access granularity. System designers must carefully consider these trade-offs based on the specific requirements and characteristics of the system to achieve optimal performance and efficiency.

Page Table Size:

Smaller Page Size: Smaller page sizes result in larger page tables because more pages are needed to cover the entire address space. This can increase memory overhead and page table management complexity.

Larger Page Size: Larger page sizes reduce the size of the page table because fewer pages are needed to cover the address space. This reduces memory overhead and simplifies page table management.

10. What are the trade-offs involved in choosing the size of virtual memory pages?

Memory Utilisation:

Smaller Page Size: Smaller page sizes reduce internal fragmentation because they allow for more precise allocation of memory. However, they may increase external fragmentation as smaller free memory blocks are more likely to be fragmented.

Larger Page Size: Larger page sizes increase internal fragmentation because some portion of each page may remain unused, leading to wasted memory. However, they reduce external fragmentation because fewer large pages are easier to manage and allocate contiguously.

10. What are the trade-offs involved in choosing the size of virtual memory pages?

Translation Lookaside Buffer (TLB) Efficiency:

Smaller Page Size: Smaller page sizes result in a higher TLB miss rate because more TLB entries are needed to cover the same amount of virtual memory space. This can lead to more frequent TLB misses and slower memory access times.

Larger Page Size: Larger page sizes reduce the TLB miss rate because fewer TLB entries are needed to cover the same amount of virtual memory space. This can improve TLB efficiency and reduce memory access times.

10. What are the trade-offs involved in choosing the size of virtual memory pages?

I/O Operations:

Smaller Page Size: Smaller page sizes may increase the number of I/O operations required for paging because smaller pages result in more frequent page faults and page swaps. This can increase disk I/O overhead and decrease overall system performance.

Larger Page Size: Larger page sizes reduce the number of page faults and page swaps because fewer pages are accessed and swapped. This can decrease disk I/O overhead and improve overall system performance.

10. What are the trade-offs involved in choosing the size of virtual memory pages?

Memory Access Granularity:

Smaller Page Size: Smaller page sizes provide finer-grained memory access control and sharing because smaller portions of memory can be mapped independently. This can improve memory access flexibility but may result in higher memory management overhead.

Larger Page Size: Larger page sizes provide coarser-grained memory access control and sharing because larger portions of memory are mapped together. This can reduce memory management overhead but may limit flexibility in memory access control and sharing.

Closing Remarks and Announcement

- **Next Week (Week 11):**

Revision week

The lab sessions and the lecture review session are optional Q&As.
Check-in will not be observed.



Thanks